

Spécification et preuve de programmes

Devoir 2 2025-26

Alain Giorgetti

Master d'Informatique de l'Université Marie et Louis Pasteur

Les questions qui commencent par le symbole  doivent être traitées dans un fichier WhyML unique, nommé `devoir2spp.mlw`, commençant par un commentaire (entre `(*` et `*)`) donnant le nom et le prénom de l'étudiant.e. Dans ce fichier, des commentaires entre `(*` et `*)` doivent indiquer le numéro de chaque question traitée, par exemple

`(* Exercice 2 question 3 *)`

juste avant la réponse à la question 3 de l'exercice 2.

La preuve et l'exécution du contenu complet de ce fichier doivent être possibles avec l'interface en ligne de Why3 (<https://www.why3.org/try/>). Dans ce but, les parties incorrectes ou incomplètes doivent être entourées par `(*` et `*)`. Ce fichier doit être déposé sur Moodle dans le devoir prévu à cet effet.

Afin que ce devoir serve aussi d'entraînement aux examens, il est demandé que toutes les réponses soient rédigées **à la main** par l'étudiant.e, sur des feuilles au format A4, avec le nom et le prénom de l'étudiant.e en haut de chaque page. Ceci inclut les réponses aux questions qui commencent par le symbole . Il s'agit alors de recopier **à la main** le code tapé précédemment dans le fichier `devoir2spp.mlw`. Ces feuilles manuscrites doivent être numérisées au format PDF, par scan ou photographie lisible, et déposées sur Moodle dans le devoir prévu à cet effet.

Un barème est donné à titre indicatif.

1 Table de vérité (1 point)

 Proposer et prouver un but ou un lemme WhyML qui permet de vérifier que le connecteur logique `&&` du langage WhyML, vu comme une fonction sur le type `bool`, est conforme à la table de vérité du "et" logique.

2 Différences entre deux tableaux (14 points)

Les réponses aux questions  sont à donner dans le fichier `devoir2spp.mlw`.

1. Expliquer par des phrases claires et simples ce que fait la fonction `search`, définie comme suit :

```
use int.Int
use ref.Ref
use array.Array

let search (a : array int) (x : int) : int =
  let ref i = 0 in
  let ref index = -1 in
  while i < a.length && index = -1 do
    if a[i] = x then index := i;
    i := i+1
  done;
  index
```

2.  Spécifier cette fonctionnalité en WhyML, le plus complètement possible, par la précondition la plus générale possible et la postcondition la plus précise possible.
3.  Proposer des invariants et un variant de boucle WhyML pour démontrer automatiquement ce contrat et la terminaison de cette fonction `search`.
4. La fonction du listing 1 stocke au début du tableau `c` les éléments présents dans le tableau `a` qui ne sont pas dans le tableau `b`. Cette fonction retourne aussi le nombre d'éléments ainsi stockés dans le tableau `c`.

```
let minus (a b c : array int) : int =
  let ref i = 0 in
  for j = 0 to a.length-1 do
    if search b a[j] = -1 then begin
      c[i] ← a[j];
      i := i+1
    end
  done;
  i
```

Listing 1 – Fonction `minus`.

Donner un exemple de tableaux `a` et `b` avec au moins quatre éléments, tel que l'entier retourné par `(minus a b c)` soit supérieur ou égal à 2. Donner aussi le contenu du tableau `c` après cette exécution, dans cet exemple.

5.  Proposer un contrat WhyML pour cette fonction `minus`, avec la précondition la plus générale possible et la postcondition la plus précise possible.
6.  Proposer des invariants pour la boucle de la fonction `minus`, pour pouvoir démontrer automatiquement la correction de cette fonction, selon son contrat donné en réponse à la question 5.
7.  Reproduire dans le fichier `devoir2spp.mlw` la fonction WhyML suivante, qui stocke au début du tableau `c` la différence symétrique des deux tableaux d'entiers `a` et `b`, composée des éléments présents dans le tableau `a` ou dans le tableau `b`, mais pas dans

ces deux tableaux. Cette méthode retourne aussi le nombre d'éléments ainsi stockés dans le tableau `c`.

```
let diff (a b c : array int) : int =
  let ref i = 0 in
  for j = 0 to a.length-1 do
    if search b a[j] = -1 then begin
      c[i] ← a[j];
      i := i+1
    end
  done;
  let ref m = i in
  for j = 0 to b.length-1 do
    if search a b[j] = -1 then begin
      c[m] ← b[j];
      m := m+1
    end
  done;
  m
```

Par exemple, si `a = {21,12,3,7,42,42,43,50,1}` et `b = {1,8,42,50,24,5}` avant exécution de l'instruction

```
let k = diff a b c in ..
```

et si `c` est un tableau d'entiers pré-alloué et de taille suffisante, alors on doit avoir `k = 8` et `{21,12,3,7,43,8,24,5}` au début du tableau `c` après l'exécution de cette instruction.

Proposer un contrat WhyML pour cette fonction, avec la précondition la plus générale possible et la postcondition la plus précise possible.

8.  Proposer des invariants pour les deux boucles de la fonction `diff`, pour pouvoir démontrer automatiquement la correction de cette fonction, selon son contrat donné en réponse à la question 7.

3 Longueur d'une concaténation de deux listes, avec Why3 (5 points)

Cet exercice porte sur la définition du type inductif des listes en WhyML, introduite dans le cours 4 sur les types, et sur les implémentations en WhyML des fonctions `length` et `append` introduites dans le cours 8 sur les calculs inductifs. L'exercice doit être traité sans utiliser ni consulter le contenu du fichier `list.mlw` de la bibliothèque standard de Why3.

1.  Dans le fichier `devoir2spp.mlw`, reproduire la définition du type inductif des listes du cours 4 et les définitions WhyML des fonctions `length` et `append` du cours 8.
2.  Exprimer la propriété suivante sous la forme d'un but WhyML.

$$length(append(l, m)) = length(l) + length(m) \quad (1)$$

3. Tenter la preuve de ce but avec l'interface en ligne de Why3 et expliquer le résultat obtenu.
4.  Définir une fonction-lemme WhyML qui permet d'obtenir une preuve automatique de cette propriété avec l'interface en ligne de Why3.

5.  Dans cette question, on suppose qu'il n'est pas possible d'écrire une fonction-lemme en WhyML. Proposer une postcondition pour la fonction `length` ou la fonction `append`, qui code la propriété (1) et dont la preuve est automatique avec l'interface en ligne de Why3.